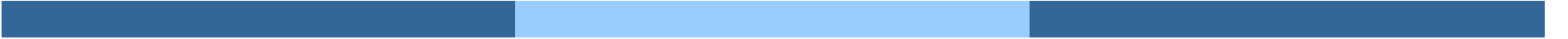


Deadlocks



Deadlocks

- ❑ Modello di Sistema
- ❑ Caratterizzazione dei deadlock
- ❑ Metodi di gestione dei deadlock
- ❑ Prevenzione deadlock
- ❑ Evitamento deadlock
- ❑ Rilevamento deadlock
- ❑ Ripristino dai deadlock

Obiettivi

- ❑ Descrivere i deadlock, che impediscono ad un insieme di thread di eseguire il loro compito
- ❑ Presentare metodi per prevenire, identificare, evitare, recuperare i deadlock

Esempio in Posix

□ Esempio di due thread in deadlock con mutex Posix

```
pthread_mutex_t first_mutex;  
pthread_mutex_t second_mutex;  
  
pthread_mutex_init(&first_mutex, NULL);  
pthread_mutex_init(&second_mutex, NULL);
```

```
/* thread_one runs in this function */  
void *do_work_one(void *param)  
{  
    pthread_mutex_lock(&first_mutex);  
    pthread_mutex_lock(&second_mutex);  
    /**  
     * Do some work  
     */  
    pthread_mutex_unlock(&second_mutex);  
    pthread_mutex_unlock(&first_mutex);  
  
    pthread_exit(0);  
}
```

```
/* thread_two runs in this function */  
void *do_work_two(void *param)  
{  
    pthread_mutex_lock(&second_mutex);  
    pthread_mutex_lock(&first_mutex);  
    /**  
     * Do some work  
     */  
    pthread_mutex_unlock(&first_mutex);  
    pthread_mutex_unlock(&second_mutex);  
  
    pthread_exit(0);  
}
```

Livelock

- ❑ Altra forma di fallimento di Liveness
- ❑ Un gruppo di thread non bloccato ma non procede
 - ❑ Continuo tentativo di eseguire un'azione che fallisce ed impedisce di avanzare
 - ▶ Es. Due persone in un corridoio che non riescono ad evitarsi

```
/* thread_one runs in this function */
void *do_work_one(void *param)
{
    int done = 0;

    while (!done) {
        pthread_mutex_lock(&first_mutex);
        if (pthread_mutex_trylock(&second_mutex)) {
            /**
             * Do some work
             */
            pthread_mutex_unlock(&second_mutex);
            pthread_mutex_unlock(&first_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&first_mutex);
    }

    pthread_exit(0);
}
```

```
/* thread_two runs in this function */
void *do_work_two(void *param)
{
    int done = 0;

    while (!done) {
        pthread_mutex_lock(&second_mutex);
        if (pthread_mutex_trylock(&first_mutex)) {
            /**
             * Do some work
             */
            pthread_mutex_unlock(&first_mutex);
            pthread_mutex_unlock(&second_mutex);
            done = 1;
        }
        else
            pthread_mutex_unlock(&second_mutex);
    }

    pthread_exit(0);
}
```

In alcuni casi si può risolvere con randomizzazione (es. periodo backoff in protocolli di rete)

Caratterizzazione Deadlock

Un deadlock avviene se le seguenti proprietà sono verificate contemporaneamente

- **Mutual exclusion:** almeno una risorsa deve essere tenuta in modalità esclusiva: solo un processo alla volta può usare la risorsa
- **Hold and wait:** almeno un thread deve mantenere almeno una risorsa ed essere in attesa di avere una risorsa aggiuntiva tenuta da altri processi
- **No preemption:** le risorse non possono essere prelevate - una risorsa può essere rilasciata solo dal processo che la detiene dopo che tale processo ha completato il suo task
- **Circular wait:** esiste un insieme $\{P_0, P_1, \dots, P_n\}$ di processi in attesa mutua, tali che P_0 è in attesa di una risorsa che è tenuta da P_1 , P_1 attende la risorsa di P_2 , ..., P_{n-1} attende la risorsa di P_n , e P_n attenda la risorsa di P_0 .

Le condizioni non sono tutte indipendenti (ultima implica hold and wait), ma è utile considerarle tutte

Modello del Sistema

- I sistemi forniscono risorse
- Tipi di risorse $R_1, R_2, \dots, R_m$
Cicli CPU, spazio di memoria, dispositivi I/O
- Ogni tipo di risorsa R_i ha W_i istanze.
- Ogni processo utilizza una risorsa come segue:
 - **request**
 - **use**
 - **release**

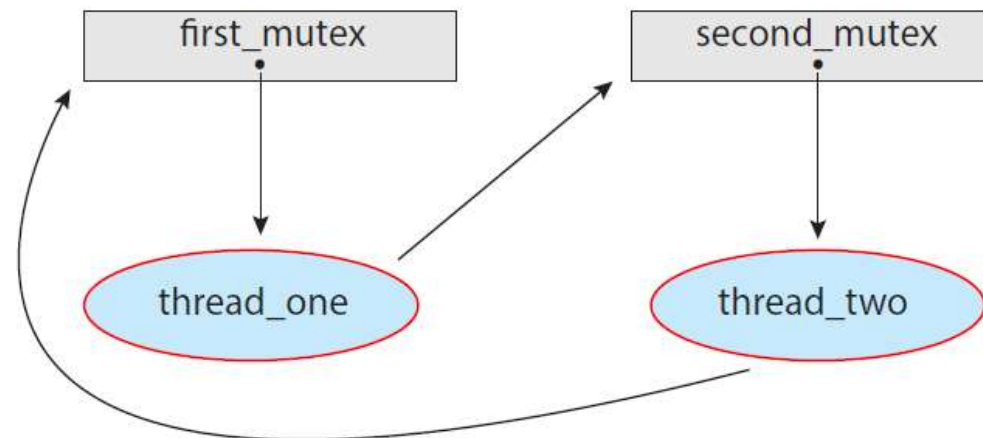
Deadlock con Lock Mutex

- Deadlock possono avvenire con system call, locking, etc.

Grafo di Allocazione delle Risorse

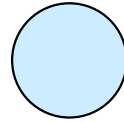
Insieme di nodi V e insieme di archi E .

- V partizionata in due tipi:
 - $P = \{P_1, P_2, \dots, P_n\}$, insieme di tutti i processi nel sistema
 - $R = \{R_1, R_2, \dots, R_m\}$, insieme di tutti i tipi di risorse del sistema
- **request edge** – archi diretti $P_i \rightarrow R_j$
- **assignment edge** – archi diretti $R_j \rightarrow P_i$

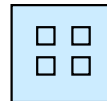


Grafo di Allocazione delle Risorse

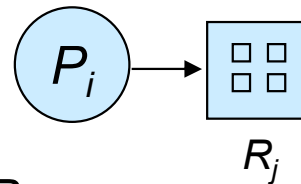
- Processo



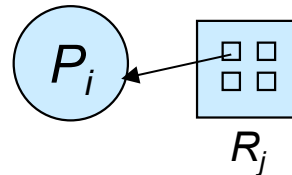
- Tipo di risorsa con 4 istanze



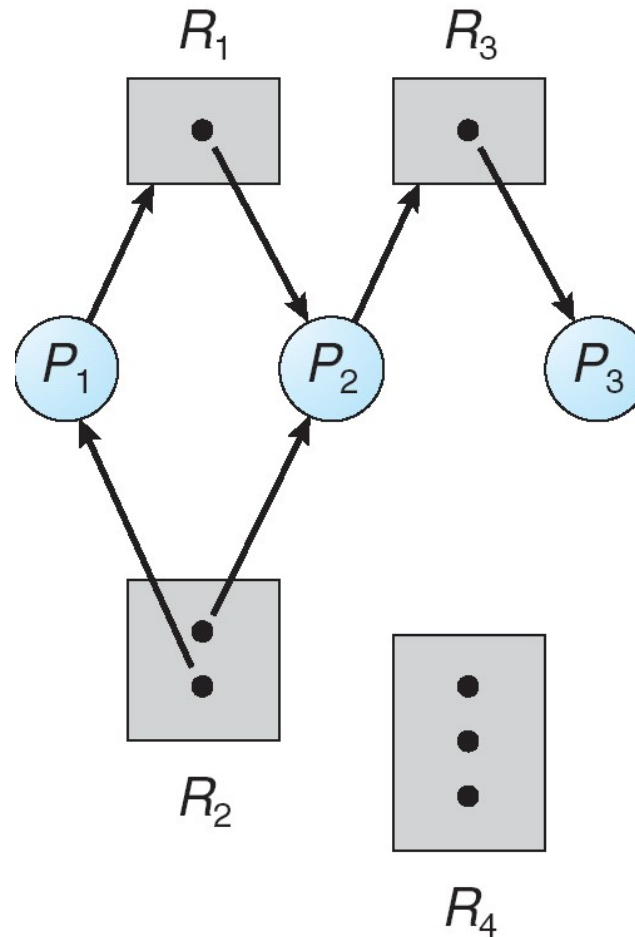
- P_i richiede istanza di R_j



- P_i detiene un'istanza di R_j



Esempio di un Grafo di Allocazione di Risorse



Senza cicli non c'è deadlock

Ciclo è condizione necessaria ma non sufficiente

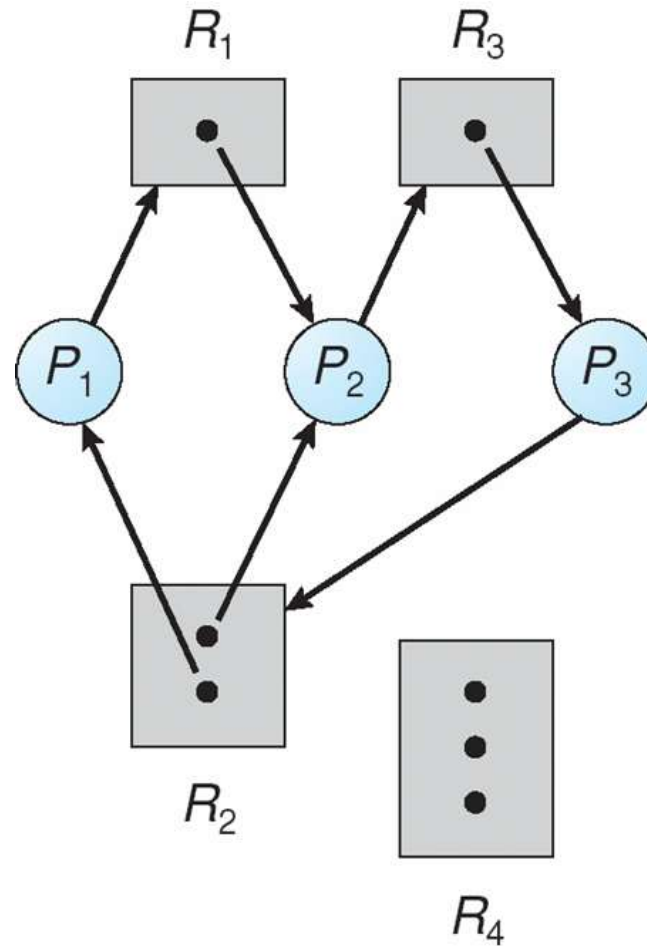
Se risorse uniche allora necessaria e sufficiente

- $T = \{T_1, T_2, T_3\}$

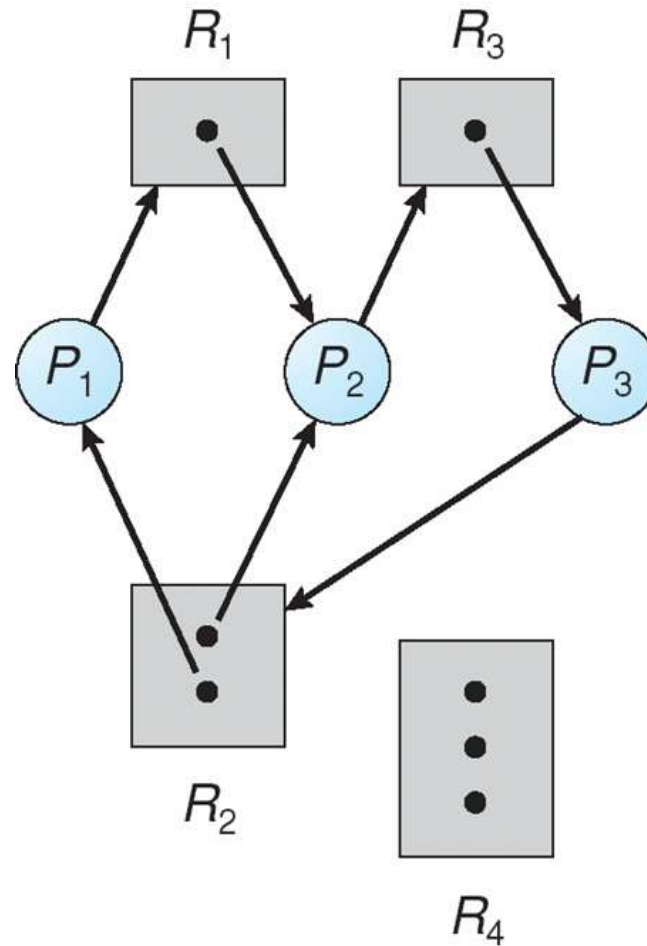
- $R = \{R_1, R_2, R_3, R_4\}$

- $E = \{T_1 \rightarrow R_1, T_2 \rightarrow R_3, R_1 \rightarrow T_2, R_2 \rightarrow T_2, R_2 \rightarrow T_1, R_3 \rightarrow T_3\}$

Esempio di un Grafo di Allocazione di Risorse



Grafo di Allocazione di Risorse con un Deadlock



Si introduca un ciclo

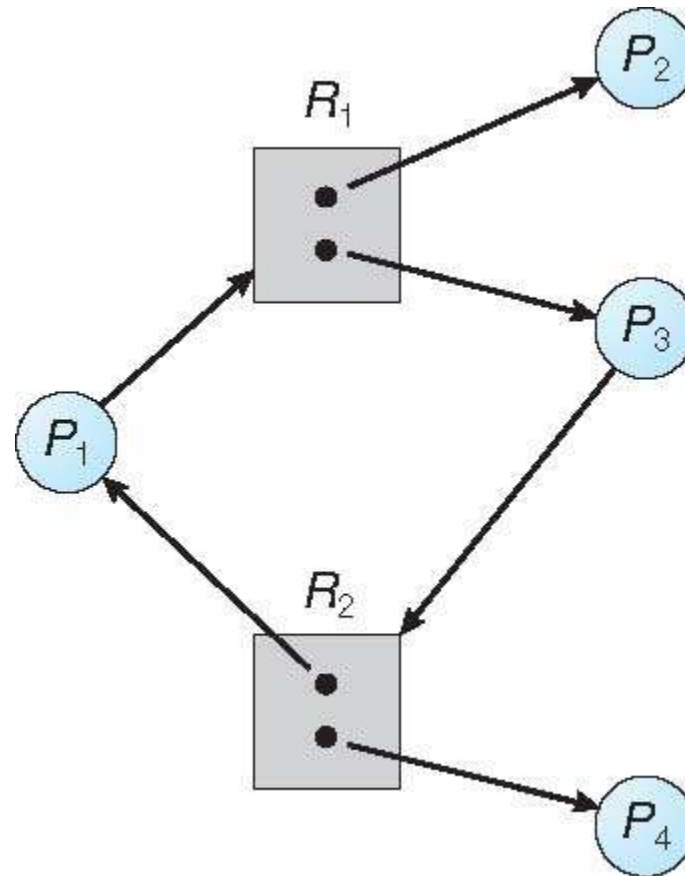
I tre thread sono in deadlock

$$\begin{array}{l} T_1 \rightarrow R_1 \rightarrow T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_1 \\ T_2 \rightarrow R_3 \rightarrow T_3 \rightarrow R_2 \rightarrow T_2 \end{array}$$

Grafo con un ciclo ma senza deadlock

... anche in questo caso ciclo

Deadlock?

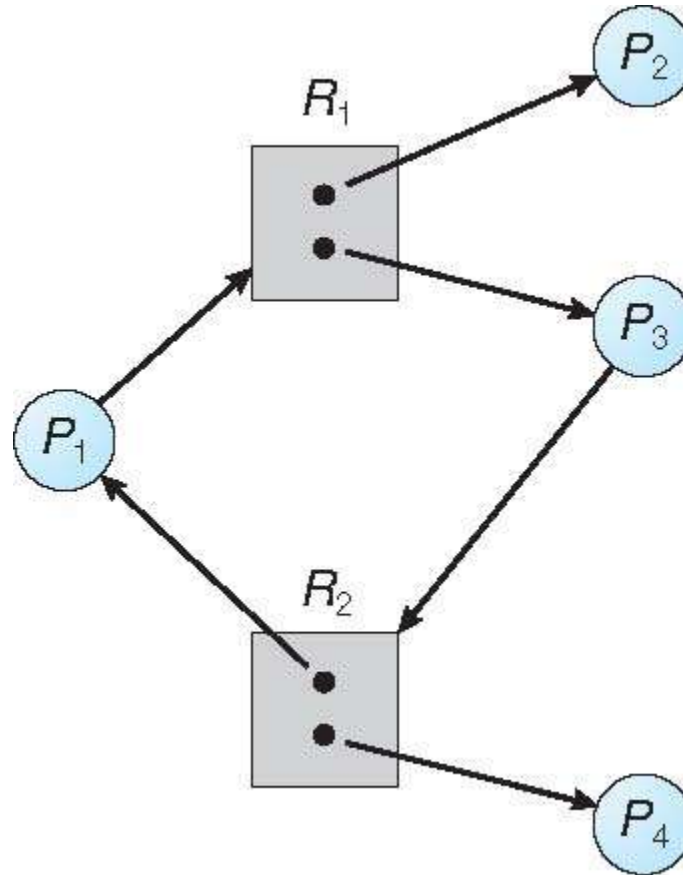


Grafo con un ciclo ma senza deadlock

... anche in questo caso ciclo

Deadlock?

P_4 può rilasciare R_2 rompendo il ciclo per consentire a P_3 l'esecuzione



Fatti di Base

- Se il grafo non ha cicli $\Rightarrow$ non ha deadlock
- Se il grafo contiene un ciclo $\Rightarrow$
 - Se esiste una sola istanza per tipo di risorsa, allora deadlock
 - Se molte istanze per tipo di risorsa, c'è possibilità di un deadlock, ma non si ha necessariamente il deadlock

Metodi di Gestione Deadlocks

Approcci al problema della gestione del deadlock

- Ignorare il problema assumendo che i deadlocks non si presentino mai nel sistema;
 - usato dalla maggior parte dei sistemi operativi, incluso UNIX
- Assicurare che il sistema non entri **mai** in un deadlock state:
 - Prevenzione di deadlock (deadlock prevention)
 - ▶ Limitare i modi in cui si fanno le richieste per evitare i deadlock
 - Evitamento del deadlock (deadlock avoidance)
 - ▶ Valutare le richieste per evitare situazioni pericolose
- Permettere al sistema di entrare in uno stato di deadlock per poi recuperare (deadlock recovery)

Prevenzione Deadlock

Limitare i modi in cui può essere fatta la richiesta tenendo presente le condizioni di deadlock

- **Mutual Exclusion** – non richiesta per risorse condivisibili (e.g., read-only files)
 - .. però non si possono limitare le richieste solo a risorse non condivisibili
- **Hold and Wait** – deve garantire che quando un processo richiede una risorsa, non detiene altre risorse
 - Imporre al processo di richiedere e allocare tutte le sue risorse prima che inizi l'esecuzione
 - Consentire al processo di richiedere risorse solo quando al processo non ne è stata allocata alcuna, le usa e le rilascia
 - Non pratico: basso utilizzo delle risorse; possibile starvation

Prevenzione Deadlock

□ Non consentire prelazione di risorse –

- Se un processo che detiene risorse richiede un'altra risorsa che non può essere immediatamente data allora tutte le risorse devono essere rilasciate
 - ▶ Le risorse prelazionate sono aggiunte alla lista delle risorse per le quali il processo è in attesa
 - ▶ Il processo verrà riavviato solo quando può ottenere le vecchie più le nuove risorse
- Oppure si cercano le risorse tra quelle di processi in attesa (che si possono prelazionare)
- Può funzionare solo per risorse facilmente recuperabili (CPU, registry, DB, etc.), non per mutex o semafori

□ Attesa circolare –

- imponi un ordine totale a tutti i tipi di risorse e richiedi che ogni processo richieda le risorse in un ordine di enumerazione crescente
- Date le risorse $R = \{R_1, R_2, \dots, R_m\}$ si assegna un numero di ordine $F(R)$
- Un processo può richiedere risorse solo rispettando l'ordine delle risorse
 - ▶ Se nuova risorsa $F(R_j) > F(R_i)$

Esempio Deadlock

```
/* thread one runs in this function */
void *do_work_one(void *param)
{
    pthread_mutex_lock(&first_mutex);
    pthread_mutex_lock(&second_mutex);
    /** * Do some work */
    pthread_mutex_unlock(&second_mutex);
    pthread_mutex_unlock(&first_mutex);
    pthread_exit(0);
}

/* thread two runs in this function */
void *do_work_two(void *param)
{
    pthread_mutex_lock(&second_mutex);
    pthread_mutex_lock(&first_mutex);
    /** * Do some work */
    pthread_mutex_unlock(&first_mutex);
    pthread_mutex_unlock(&second_mutex);
    pthread_exit(0);
}
```

$F(\text{first_mutex}) = 1$
 $F(\text{second_mutex}) = 5$

Esempio di Deadlock con Lock Ordering

```
void transaction(Account from, Account to, double amount)
{
    mutex lock1, lock2;
    lock1 = get_lock(from);
    lock2 = get_lock(to);
    acquire(lock1);
    acquire(lock2);
    withdraw(from, amount);
    deposit(to, amount);
    release(lock2);
    release(lock1);
}
```

Transaction 1 e 2 eseguite concorrentemente. Transaction 1 trasferisce \$25 da A a B mentre Transaction 2 trasferisce \$50 da B ad A

```
transaction(checking_account, savings_account, 25.0)
transaction(savings_account, checking_account, 50.0)
```

Deadlock Avoidance

Il Sistema deve avere informazione ***a priori*** aggiuntionale:

- Il modello più semplice ed utile richiede ai processi di dichiarare il ***massimo numero*** di risorse necessarie per tipo di processo
- L'algoritmo di deadlock-avoidance esamina dinamicamente lo stato di allocazione delle risorse per assicurare che non si entri mai in una situazione di circular-wait
- Lo stato di allocazione delle risorse (resource-allocation ***state***) è dato dal numero di risorse disponibile e allocate, e dalle massime richieste per processo

Stato Sicuro

- Quando un processo richiede una risorsa disponibile, il sistema deve decidere se l'allocazione immediata lascia il sistema in uno stato sicuro
- Un sistema è in uno **stato sicuro** se esiste una sequenza $\langle P_1, P_2, \dots, P_n \rangle$ di TUTTI i processi nel sistema tali che per ogni P_i , le risorse che P_i può ancora richiedere possono essere soddisfatte dalle risorse correntemente disponibili + le risorse tenute da tutti i P_j , con $j < i$
- Cioè:
 - Se le risorse richieste da P_i non sono immediatamente disponibili, allora P_i può aspettare finché tutti i P_j hanno finito
 - Quando P_j ha finito, P_i può ottenere le risorse richieste, eseguirle, rilasciando le risorse allocate e terminare
 - Quando P_i termina, P_{i+1} può ottenere le risorse necessarie, e così via
...

Stato Sicuro

- Esempio: 12 risorse e 3 thread

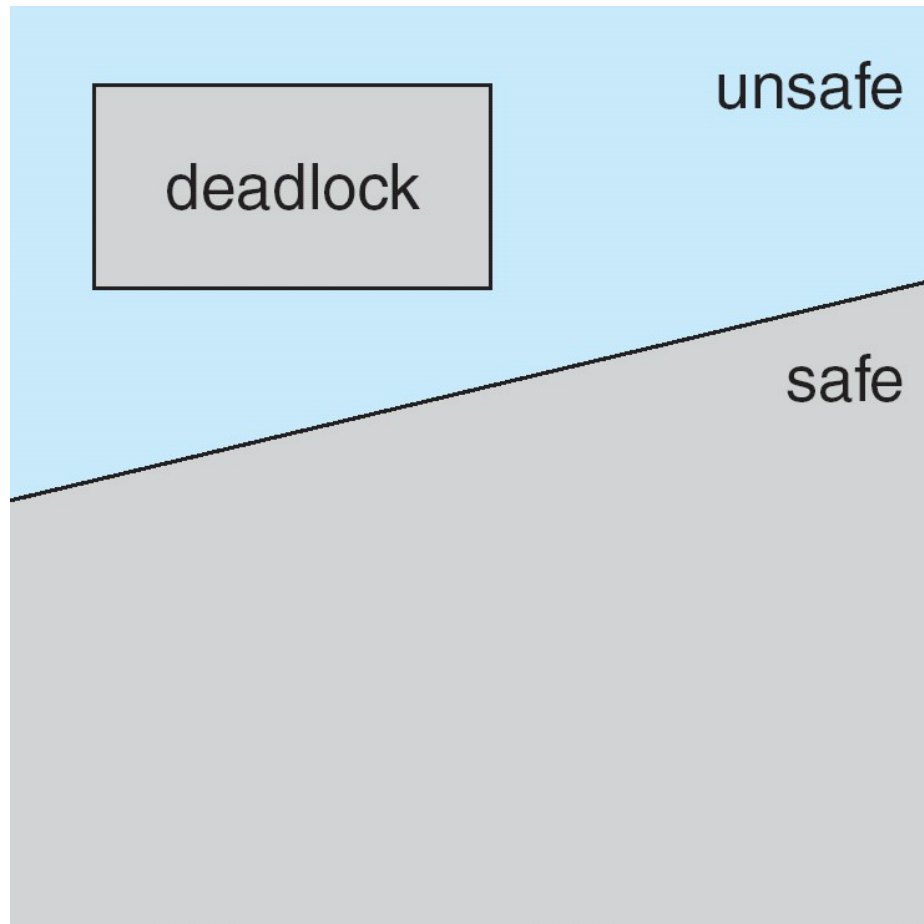
	<u>Maximum Needs</u>	<u>Current Needs</u>
T_0	10	5
T_1	4	2
T_2	9	2

- Il Sistema è in stato sicuro
 - Allocate 9 ne rimangono 3
 - La sequenza $\langle T_1, T_0, T_2 \rangle$ soddisfa il requisito
 - ▶ T_1 ne prende 2 e restituisce 4
 - ▶ T_0 ne prende 5 e restituisce 10
 - ▶ T_2 ne prende 7 e finisce
- Se invece si alloca a T_2 un'ulteriore risorsa al tempo t_1 lo stato non è più safe

Fatti di Base

- Se un sistema è in stato sicuro $\Rightarrow$ non deadlock
- Se un sistema è in stato non sicuro $\Rightarrow$ possibilità di deadlock
- Evitamento $\Rightarrow$ assicurare che un sistema non entri mai in uno stato non sicuro

Safe, Unsafe, Deadlock State



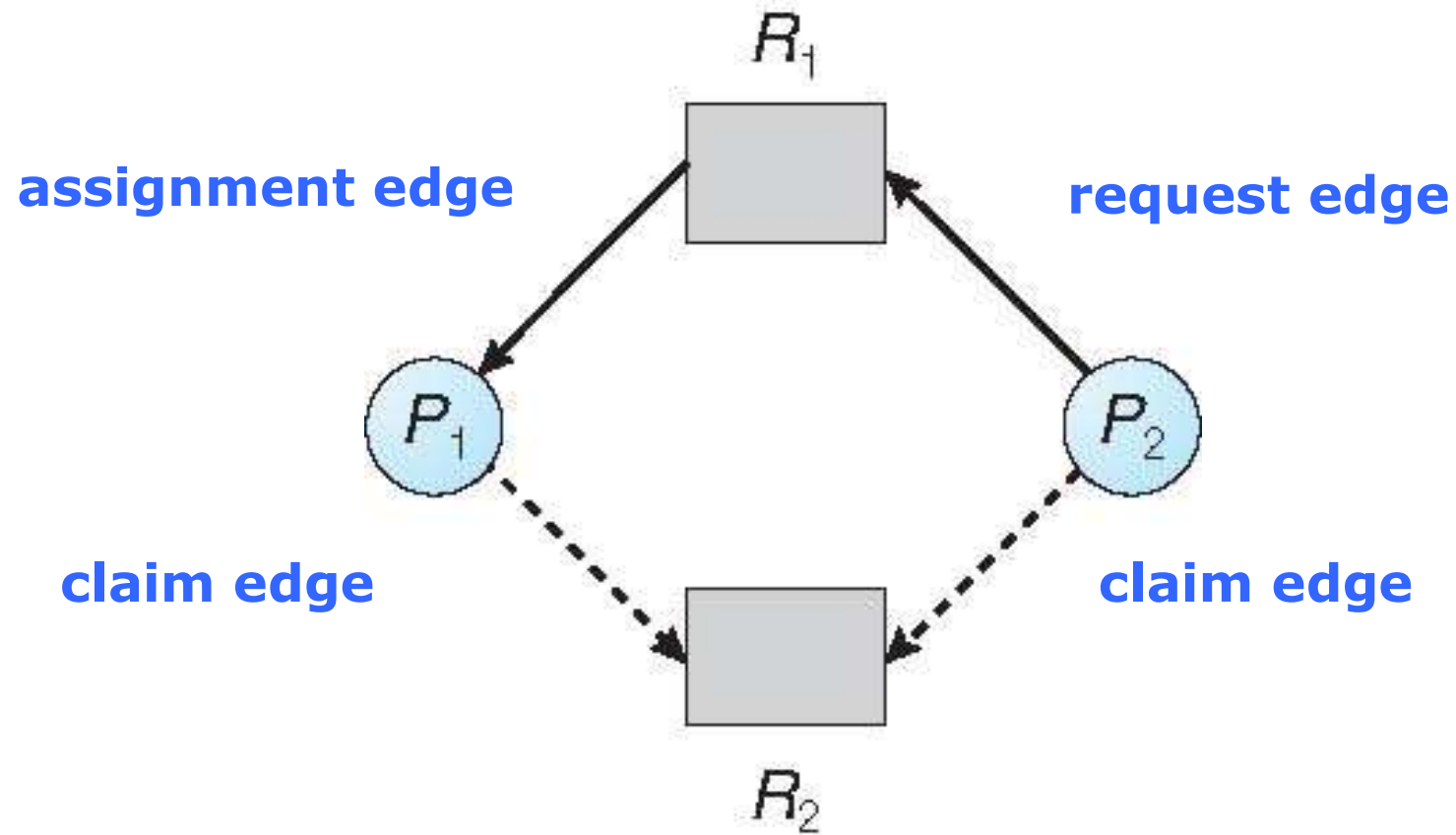
Algoritmo di Avoidance

- Singola istanza di un tipo di risorsa
 - Usa un resource-allocation graph
- Multiple istanze di tipo di risorsa
 - Usa l'algoritmo del banchiere

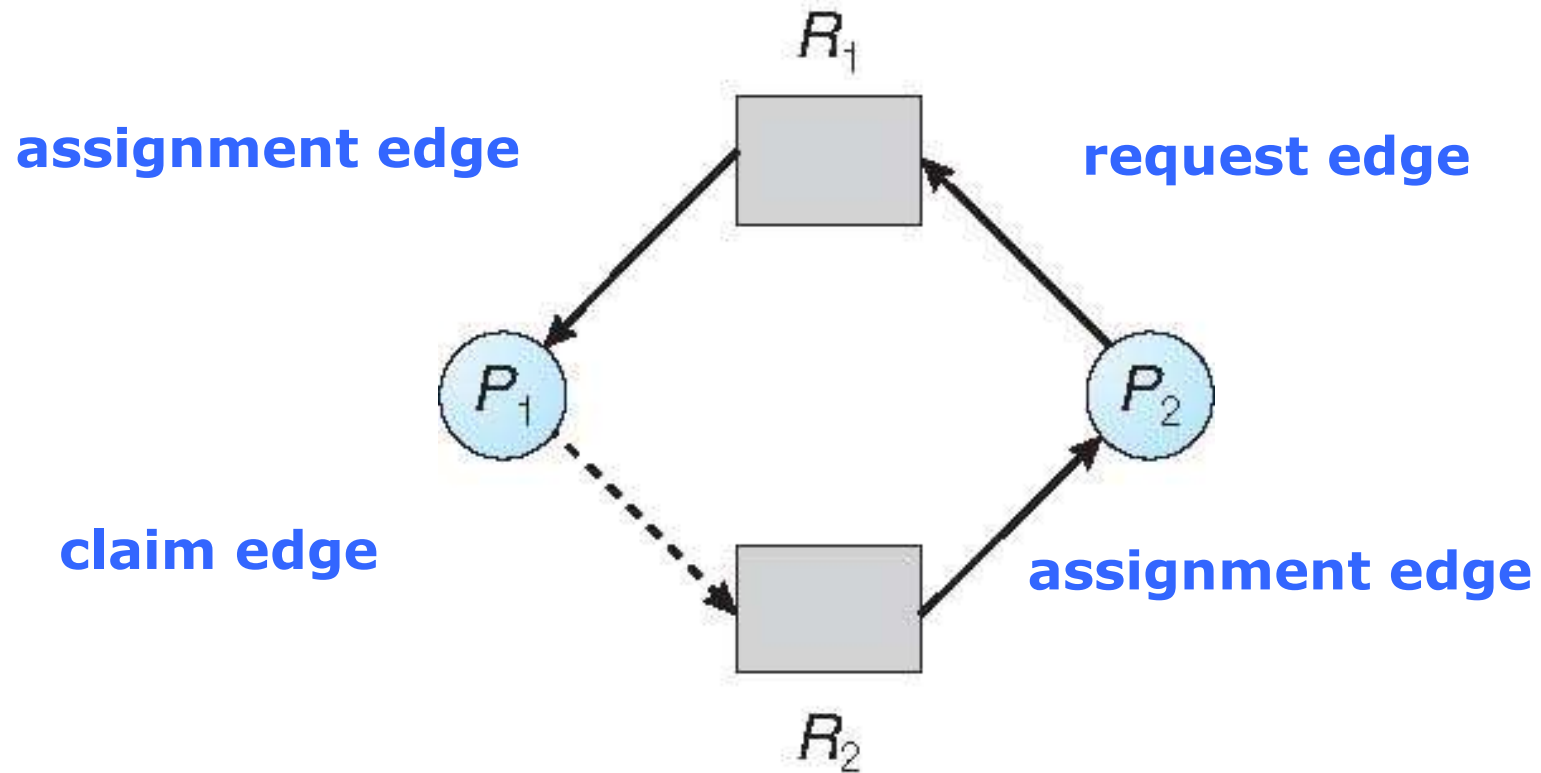
Schema con grafo di allocazione

- Si introduce la **claim edge** $P_i \rightarrow R_j$ che indica che il processo P_i potrebbe richiedere la risorsa R_j ; (linea tratteggiata)
- La claim edge si converte in **request edge** quando un processo richiede la risorsa
- La request edge si converte in **assignment edge** quando la risorsa è allocata al processo
- Quando la resource è rilasciata dal processo, l'assignment edge si riconverte in claim edge
- Le risorse devono essere richieste *a priori* nel sistema

Resource-Allocation Graph



State unsafe in Resource-Allocation Graph



Algoritmo di Resource-Allocation Graph

- Supponi che il processo P_i richieda una risorsa R_j
- La richiesta può essere garantita solo se trasformando l'arco di richiesta in un assegnamento non porta alla formazione di un ciclo nel grafo di allocazione delle risorse
- Altrimenti la richiesta viene messa in attesa
- Complessità di rilevare un ciclo è n^2 con n numero di thread del sistema

Algoritmo del Banchiere

- ❑ Allocazione delle risorse della banca per soddisfare tutti i clienti
- ❑ Applicabile per multiple istanze
- ❑ Ogni processo deve dichiarare a priori il massimo utilizzo di risorse
- ❑ Quando un processo richiede una risorsa potrebbe dover aspettare
- ❑ Quando un processo ottiene tutte le sue risorse deve restituirle in un tempo finito

Algoritmo del Banchiere

- Quando un processo richiede un insieme di risorse, il Sistema deve controllare se l'assegnazione lascia il Sistema in uno **stato sicuro**
- Se lo stato è sicuro le risorse sono allocate altrimenti la richiesta viene messa in attesa finché qualche processo libera delle risorse aggiuntive
- Si introducono diverse strutture dati per rappresentare lo stato del sistema

Strutture Dati per l'Algoritmo del Banchiere

Sia n = numero di processi ed m = numero di tipi di risorse.

- **Available:** Vettore di lunghezza m . Se $available[j] = k$, ci sono k istanze della risorsa di tipo R_j disponibili
- **Max:** matrice $n \times m$. Se $Max[i,j] = k$, allora il processo P_i potrebbe richiedere al massimo k istanze della risorsa di tipo R_j
- **Allocation:** matrice $n \times m$. Se $Allocation[i,j] = k$ allora P_i ha correntemente allocate k istanze di R_j
- **Need:** matrice $n \times m$. Se $Need[i,j] = k$, allora P_i potrebbe aver bisogno di k più istanze di R_j per complare il suo task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Esempio Stato Sicuro

- Esempio precedente: 12 risorse (tutte di un tipo) e 3 thread
- Matrici Max e Allocation

	<u>Maximum Needs</u>	<u>Current Needs</u>
T_0	10	5
T_1	4	2
T_2	9	2

- Il Sistema era in stato sicuro
 - Allocate 9 rimangono 3
 - La sequenza $\langle T_1, T_0, T_2 \rangle$ soddisfa il requisito

Esempio Algoritmo Banchiere

- 5 processi da P_0 a P_4 ;

3 tipi di risorse:

A (10 istanze), B (5 istanze) e C (7 istanze)

- Stato al tempo T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Algoritmo di Safety

1. Siano **Work** e **Finish** vettori di lunghezza m (risorse) ed n (processi), rispettivamente. Inizializza:

Work = Available

Finish [i] = false for $i = 0, 1, \dots, n-1$

2. Trova un indice i tale che entrambi:

(a) **Finish [i] = false**

(b) **Need _{i} ≤ Work**

Trova un processo per cui le assegnazioni non siano state soddisfatte

Se non esiste i , vai al passo 4

3. **Work = Work + Allocation _{i}**

Finish[i] = true

vai al passo 2

Aggiunge le disponibilità del processo i -esimo assumendolo concluso

4. Se **Finish [i] == true** per tutti gli i , allora il sistema è in uno stato sicuro (safe state)

L'algoritmo verifica se lo stato è sicuro e può richiedere un ordine di $m \times n^2$ operazioni

Algoritmo di Richiesta di Risorsa per il Processo P_i

Algoritmo per determinare se una richiesta può essere accordata in sicurezza

$Request_i$ = vettore richieste per il processo P_i .

Se **$Request_i[j] = k$** allora il processo P_i vuole k istanze delle risorse di tipo R_j

1. Se **$Request_i \leq Need_i$** vai al passo 2. Altrimenti, segnala la condizione di errore dal momento che il processo ha superato il suo massimo dichiarato
2. Se **$Request_i \leq Available$** vai al passo 3. Altrimenti P_i deve aspettare perché le risorse non sono disponibili
3. Prova ad allocare le risorse richieste per P_i modificando lo stato come segue:

$$Available = Available - Request_i;$$

$$Allocation_i = Allocation_i + Request_i;$$

$$Need_i = Need_i - Request_i;$$

- Verifica se safe
- Se safe $\Rightarrow$ le risorse sono allocate a P_i
- Se unsafe $\Rightarrow P_i$ deve aspettare e il vecchio stato di allocazione delle risorse viene recuperato

Esempio Algoritmo Banchiere

- 5 processi da P_0 a P_4 ;

3 tipi di risorse:

A (10 istanze), B (5 istanze) e C (7 istanze)

- Stato al tempo T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	$A \ B \ C$	$A \ B \ C$	$A \ B \ C$
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Esempio

- Il contenuto della matrice **Need** è definito come **Max – Allocation**

	<u>Need</u>		
	A	B	C
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- Il Sistema è in un stato sicuro (safe state) perché la sequenza $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ soddisfa i criteri di safety

Esempio

- Il contenuto della matrice **Need** è definito come **Max – Allocation**

	<u>Need</u>	<u>Available</u>
	A B C	3 3 2
P_0	7 4 3	<u>Work</u>
P_1	1 2 2	3 3 2
P_2	6 0 0	<u>Finish</u>
P_3	0 1 1	F F F F F
P_4	4 3 1	

- Per P_1 la $Need \leq Work$ e $Finish[1] = F$
- $Work = Work + Allocation[1] = (2\ 1\ 0) + (3\ 2\ 2) = 5\ 3\ 2$, $Finish[1] = T$
- Per P_3 la $Need \leq Work$ e $Finish[3] = F$
- $Work = Work + Allocation[3] = 5\ 4\ 3$, $Finish[1] = T$
- ...

Esempio: P_1 Richiede (1,0,2)

- Controlla che $\text{Request} \leq \text{Available}$ (cioè, $(1,0,2) \leq (3,3,2) \Rightarrow \text{true}$

	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Eseguendo l'algoritmo di safety si vede che la sequenza $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ soddisfa i criteri
- Può la richiesta per (3,3,0) di P_4 essere garantita?
- Può la richiesta per (0,2,0) di P_0 essere garantita?

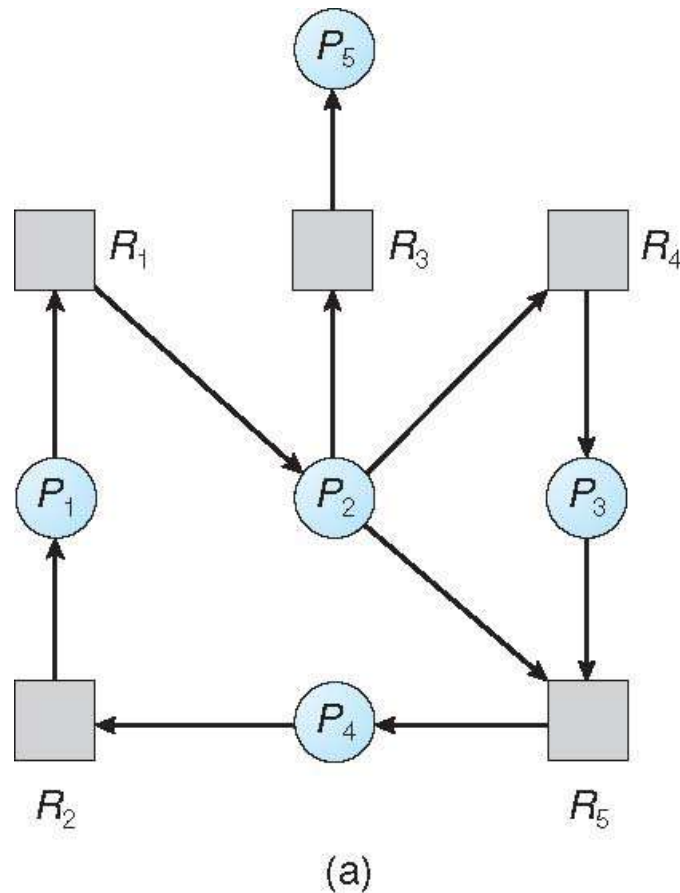
Rilevazione Deadlock

- Permetti al sistema di entrare nello stato di deadlock
- Servono quindi:
 - Algoritmi di deadlock detection
 - Metodi di deadlock recovery
- Costi:
 - Costo del monitoraggio (strutture dati ed algoritmi)
 - Potenziale perdita di dati durante il recovery
- Consideriamo due casi:
 - Istanza singola di risorsa
 - Istanze multiple di risorsa

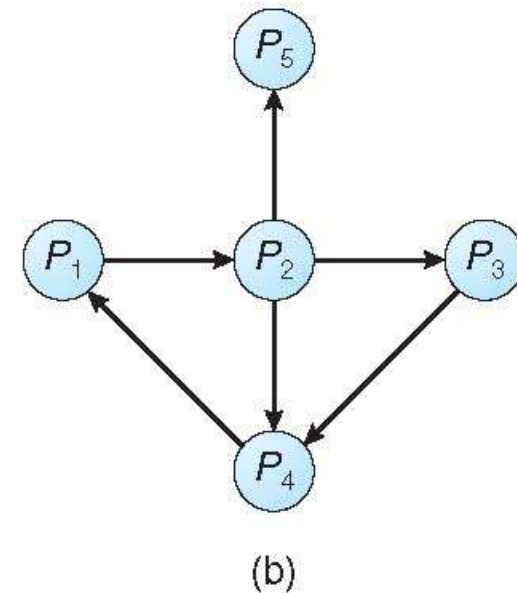
Istanza Singola di Ogni Tipo di Risorsa

- Si usa una variante del grafo di allocazione delle risorse chiamato grafo **wait-for** (grafo delle attese)
 - Si rimuovono i nodi delle risorse che si bypassano con archi tra processi
 - Nodi sono solo per i processi
 - $P_i \rightarrow P_j$ se P_i sta aspettando P_j

Grafo di Allocazione delle e Grafo delle Attese



Grafo di allocazione delle risorse



Grafo delle Attese corrispondente

Istanza Singola di Ogni Tipo di Risorsa

- Si usa una variante del grafo di allocazione chiamato grafo **wait-for** (grafo delle attese)
 - Si rimuovono i nodi delle risorse che si bypassano con archi tra processi
 - Nodi sono solo processi
 - $P_i \rightarrow P_j$ se P_i sta aspettando P_j
- Periodicamente invoca un algoritmo che cerca i cicli nel grafo. Se c'è ciclo allora c'è un deadlock
- Un algoritmo per rilevare un ciclo in un grafo richiede un ordine di n^2 operazioni, dove n è il numero di vertici nel grafo
- Esempio
 - BCC toolkit su Linux mette a disposizione un `deadlock_detector` che traccia l'uso dei `pthread_mutex_lock` e `pthread_mutex_unlock` costruendo un grafo delle attese e segnalando i deadlock

Molte Istanze di un Tipo di Risorsa

Nel caso di più istanze occorrono più strutture dati come per l'algoritmo del banchiere

- **Available:** un vettore di lunghezza m che indica il numero di risorse disponibili per ogni tipo di risorsa
- **Allocation:** una matrice $n \times m$ definisce il numero di risorse per tipo correntemente allocato ad ogni processo
- **Request:** una matrice $n \times m$ indica la richiesta corrente di ogni processo. Se **Request** $[i][j] = k$, allora il processo P_i sta richiedendo k più istanze di risorsa di tipo R_j .

Algoritmo di Rilevamento

1. Siano **Work** e **Finish** vettori di lunghezza **m** ed **n**

Inizializza:

(a) **Work = Available**

(b) For $i = 1, 2, \dots, n$, if **Allocation_i** $\neq 0$, then
Finish[i] = false; otherwise, **Finish[i] = true**

2. Trova un indice **i** tale che entrambi:

(a) **Finish[i] == false**

(b) **Request_i ≤ Work**

Se non esiste tale **i**, vai al passo 4

Algoritmo di Rilevamento

3. **$Work = Work + Allocation_i$**
 $Finish[i] = true$
vai al passo 2

Rilasciate le risorse dopo il completamento, si assume che non siano necessarie altre risorse da allocare per finire il processamento

4. Se **$Finish[i] == false$** , per qualche i , $1 \leq i \leq n$, allora il sistema è in uno stato di deadlock. Inoltre, se **$Finish[i] == false$** , allora P_i è in deadlocked

Algoritmo richiede un ordine di $O(m \times n^2)$ operazioni per rilevare se il sistema è in stato di deadlock

Esempio di Algoritmo di Rilevamento

- Cinque processi da P_0 a P_4 ; tre tipi di risorsa A (7 istanze), B (2 istanze), and C (6 istanze)

- Stato al tempo T_0 :

	<u>Allocation</u>	<u>Request</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

- La sequenza $\langle P_0, P_2, P_3, P_1, P_4 \rangle$ porterà a ***Finish[i] = true*** per tutti le i

Esempio

- P_2 richiede un'istanza aggiuntiva di tipo **C**

	<u>Request</u>		
	A	B	C
P_0	0	0	0
P_1	2	0	2
P_2	0	0	1
P_3	1	0	0
P_4	0	0	2

- Stato del sistema?
 - Può rilasciare le risorse tenute dal processo P_0 , ma le risorse sono insufficienti per soddisfare le richieste degli altri processi
 - Esiste un deadlock per i processi P_1 , P_2 , P_3 , e P_4

Uso dell'Algoritmo di Rilevamento

- Quanto spesso invocare il rilevamento?
- Dipende da:
 - Quanto spesso un deadlock è probabile che avvenga?
 - ▶ Se frequente va invocato frequentemente
 - Può coinvolgere più processi?
 - ▶ Nel caso estremo può essere invocato ogni volta che una richiesta non può essere soddisfatta
 - ▶ In questo caso si può identificare anche chi ha causato il deadlock
 - ▶ Però dispendioso, quindi check ad intervalli fissi
 - Quanti processi sarà necessario recuperare?
 - ▶ uno per ogni ciclo disgiunto
- Se l'algoritmo di detection invocato ad intervalli definiti (o quando l'uso della CPU va sotto una soglia definita)
 - possibili molti cicli nel grafo delle risorse
 - ▶ complicato risalire a quali dei tanti processi abbia “causato” il deadlock.

Ripristino dal Deadlock

- Quando viene trovato un deadlock va gestito:
 - Avvertire operatore che gestisce a mano
 - Gestione automatica

- Due modalità principali di gestione:
 - Terminare uno o più processi coinvolti
 - Prelazionare risorse dai processi coinvolti nel deadlock

Ripristino dal Deadlock: Terminazione di Processo

□ Terminazione

□ Abort di tutti i processi in deadlock

- ▶ Molto dispendioso tutta la computazione persa

□ Abort dei processi, uno alla volta finché il ciclo di deadlock cycle è eliminato

- ▶ Invocato il deadlock check ad ogni passo (overhead)

□ In quale ordine dovremmo sceglierli per fare l'abort?

- ▶ Si dovrebbe scegliere il costo minimo, ma diversi criteri:

1. Priorità del processo
2. Per quanto tempo ha lavorato e quanto manca al completamento
3. Quante e quali risorse il processo ha già usato (si possono prelazionare?)
4. Quante risorse il processo necessita per completare
5. Quanti processi necessiteranno di essere terminati
6. È un processo interattivo o batch

Ripristino dal Deadlock: Prelazione di Risorse

- Si selezionano risorse da prelazionare finché il deadlock non è risolto
- Per la selezione della risorsa occorre:
 - **Selezionare una vittima** – quale risorsa e quale processo prelazionare?
 - ▶ Minimizza il costo: il numero di risorse trattenute, il tempo di computazione impiegato, etc.
 - **Rollback** – se prelazionata risorsa da un processo che farne del processo?
 - ▶ Deve ritornare in qualche safe state e ripartire da quello stato. Metodo più semplice ripartire da capo. Altrimenti da stato intermedio di computazione, però va mantenuto.
 - **Starvation** – come evitare lo starvation?
 - ▶ Se c'è una cost function per la scelta, sempre lo stesso processo può essere scelto come vittima. Per evitarlo si può includere il numero dei rollback nei fattori di costo

Riassunto

- Definizione di deadlock
- Condizioni per avere un deadlock
- Grafi di allocazione delle risorse e cicli
- **Prevenzione dei deadlock** in base alle condizioni di deadlock
- Metodi di **evitamento del deadlock** per risorse singole (grafo delle allocazioni) e multiple (algoritmo del banchiere)
- Metodi di **deadlock detection** per risorse singole e multiple
- Metodi di **deadlock recovery**